

Software Requirements Engineering (SWE3104)
Instructor: Behailu Getachew (Asst. Prof.)



Chapter 6-Requirements Effort estimation & Project Velocity



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Introduction**
- **Project Effort Estimation**
 - **Categorization of Estimation methods:**
- **Early models of effort estimation (LOCs based).**
- **Estimating in Agile methodologies**
 - **Project Velocity**

Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity



- **Introduction**

- **Fundamental Estimation Questions**

- How much effort is required to complete an activity?
 - How much calendar time is needed to complete an activity?
 - What is the total cost of an activity?



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Introduction ...**
 - **Software cost components**
 - Hardware and software costs
 - Travel and training costs
 - Effort costs (the dominant factor in most projects):
 - salaries of engineers involved in the project
 - Social and insurance costs
 - Effort costs must take overheads into account:
 - costs of building, heating, lighting
 - costs of networking and communications
 - costs of shared facilities (e.g library, staff restaurant, etc.)

Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity



- **Project Effort Estimation**

- Project Effort Estimation is the process of forecasting or approximating the time and cost of completing project deliverables or the task of balancing the expectations of stakeholders and the need for control while the project is implemented.

Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity



- **Project Effort Estimation ...**
 - **Categorization of effort estimation methods**
 - **Expert estimation methods**
 - are based on the knowledge and experience of the experts working on the software to provide the estimates.
 - **Algorithmic models**
 - Examples: COCOMO in its different versions, Use-Case-based estimation, function point analysis methods.

Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity



- **Project Effort Estimation ...**
 - **Categorization of effort estimation methods ...**
 - **Planning Poker**
 - It is a widely used effort estimation method in Agile software development.
 - Planning Poker is based on expert judgment and involves the whole development team in the process.
 - As such, it is a technique to combine the opinion of multiple experts in order to create an effort estimate.

Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity



- **Project Effort Estimation ...**
 - **Categorization of effort estimation methods ...**
 - **Planning Poker**
 - It is a method used to manage the **negotiation** between the **engineering team** (“Development”) and **the stakeholders** (“Business”).
 - It gains some **emotional distance from the planning process** by treating it as a game, where the **playing pieces** are “**user stories**” written on index cards and the **goal** is to **assign value** to **stories** and put them into **production over time**.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Project Effort Estimation ...**
 - **Categorization of effort estimation methods ...**
 - **Machine learning**
 - Examples: Analogy, Nearest Neighbor Algorithms, Goal directed preference (cases with similar goals are selected), specificity preference (cases with exact matches on some features are selected over general matches).



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Project Effort Estimation ...**

- The general form of effort estimation in any model can be: **$E = aS^b$**
 - where '**E**' is effort, '**S**' is size typically measured as lines of code (LOC) or function points, '**a**' is a productivity parameter and '**b**' is an economies or diseconomies of scale parameter.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Project Effort Estimation ...**

- A fundamental **problem** of software effort estimation is the **determination of software size**.
- The different approaches to measure software size are:
 - **Line of Code (LOC), function point (FP), use case point (UCP).**



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based).**
 - **LOC-based models**
 - A widely known model based on LOC is the constructive cost model (COCOMO). (*Boehm, 1981*).
 - COCOMO classifies projects into three broad categories: **organic** or **simple**, **semidetached** or **average**, and **embedded** or **complex**.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based) ...**
 - **COCOMO**
 - The intermediate version of COCOMO first calculates a nominal effort estimate in worker-months (WM) using a non-linear function based on the size of the software measured in thousands of **delivered source instructions** (KDSI):
 - **$WM = \alpha(KDSI)^\beta$**
 - where the values of the constants **α** and **β** are different for **organic**, **semidetached**, and **embedded** projects.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based). ...**
 - **COCOMO ...**
 - Next, it adjusts the nominal estimate by multiplying **WM** by the ratings on **15 "cost drivers"** that include **attributes** of the **product, computer, personnel**, and **project** (see the able on slide 34-37).



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based) ...**
 - **COCOMO ...**
 - Cocomo (**Constructive Cost Model**) is a **regression model** based on **LOC**, i.e **number of Lines of Code**.
 - It is a **procedural cost estimate model** for software projects and is often used as a process of reliably predicting the **various parameters** associated with making a project such as **size, effort, cost, time**, and **quality**.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based)**
 - **COCOMO ...**
 - The key parameters which define the quality of any software products, which are also an outcome of the Cocomo are **primarily Effort & Schedule:**
 - **Effort:** Amount of labor that will be required to complete a task. It is measured in person-months units.
 - **Schedule:** Simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, months.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based) ...**
 - **COCOMO ...**
 - Different models of Cocomo have been proposed to predict the cost estimation at different levels, based on the amount of accuracy and correctness required.
 - Example: **Boehm's** definition of organic, semidetached, and embedded systems.
 - **Organic** – A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based) ...**
 - **COCOMO ...**
 - Example: **Boehm's** definition of organic, semidetached, and embedded systems ...
 - **Semi-detached** – A software project is said to be a **Semi-detached** type if the vital characteristics such as **team size, experience, knowledge** of the various programming environment lie in between that of **organic** and **Embedded**.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Early models of effort estimation (LOCs based) ...**
 - **COCOMO ...**
 - Example: **Boehm's** definition of organic, semidetached, and embedded systems ...
 - **Semi-detached ...**
 - A familiar and difficult to develop compared to the **organic ones** and require **more experience** and **better guidance** and **creativity**.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **COCOMO ...**
 - Example: **Boehm's** definition of **organic**, **semidetached**, and **embedded systems ...**
 - **Embedded:**
 - A **software project** requiring the **highest level of complexity, creativity**, and **experience requirement** fall under this category.
 - Such software requires a **larger team size** than the **other two models** and also **the developers** need to be **sufficiently experienced** and creative to **develop such complex models**.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **COCOMO ...**
 - All the above **system types** utilize different values of the constants used in **Effort Calculations**.
 - **Types of Models:** COCOMO consists of a **hierarchy of three increasingly detailed and accurate forms**.
 - Any of the **three forms** can be adopted according to our **requirements**.
 - These are types of COCOMO model:
 - Basic COCOMO Model
 - Intermediate COCOMO Model
 - Detailed COCOMO Model



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **COCOMO ...**
 - Types of COCOMO model:
 - Basic COCOMO Model
 - **First Level, Basic COCOMO** can be used for quick and slightly rough calculations of Software Costs.
 - Its accuracy is somewhat restricted due to the absence of sufficient factor considerations.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **COCOMO ...**
 - **Types of COCOMO model ...:**
 - Intermediate COCOMO Model
 - **Intermediate COCOMO** takes these **Cost Drivers** into account.
 - Detailed COCOMO Model
 - **Detailed COCOMO** takes **Cost Drivers and** additionally accounts for the **influence of individual project** phases, i.e in case of **Detailed** it accounts for both these cost drivers and also calculations are **performed phase-wise** henceforth producing a **more accurate result**.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **COCOMO ...**
 - These models are further discussed below.
 - **Estimation of Effort: Calculations**
 - **Basic Model**
 - $E = a(KLOC)^b$
 - $Time = a(Effort)^d$
 - **Personrequired = Effort/Time**
 - The above formula is used for the cost estimation of for the basic COCOMO model, and also is used in the subsequent models.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Basic COCOMO**

- These models are further discussed below.
 - The constant values a, b, c and d for the Basic Model for the different categories of system:

Software Projects	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

- **The effort** is measured in **Person-Months** and as evident from the formula is **dependent** on **Kilo-Lines of code**.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Basic COCOMO ...**
 - These models are further discussed below.
 - The **development time** is measured in **months**.
 - These formulas are used as such in the **Basic Model calculations**, as **not much consideration of different factors** such as **reliability**, **expertise** is taken into account, henceforth the **estimate is rough**.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Basic COCOMO ...**

- **C++ program for Basic COCOMO**

```
// C++ program to implement basic COCOMO
#include<bits/stdc++.h>
using namespace std;

// Function for rounding off float to int
int fround(float x){
    int a;
    x=x+0.5;
    a=x;
    return(a);
}
```



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Basic COCOMO ...**

```
// Function to calculate parameters of Basic COCOMO
void calculate(float table[][4], int n, char mode[][15], int size)
{
    float effort, time, staff;
    int model;

    // Check the mode according to size
    if(size >= 2 && size <= 50)
        model = 0; // organic
    else if(size > 50 && size <= 300)
        model = 1; // semi-detached
    else if(size > 300)
        model = 2; // embedded
```



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

○ **Basic COCOMO ...**

```
cout<<"The mode is "<<mode[model];  
  
// Calculate Effort  
effort = table[model][0]*pow(size,table[model][1]);  
  
// Calculate Time  
time = table[model][2]*pow(effort,table[model][3]);  
  
//Calculate Persons Required  
staff = effort/time;  
  
// Output the values calculated  
cout<<"\nEffort = "<<effort<<" Person-Month";  
cout<<"\nDevelopment Time = "<<time<<" Months";  
cout<<"\nAverage Staff Required = "<<round(staff)<<" Persons"; }
```



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

○ **Basic COCOMO ...**

```
int main(){  
    float  
    table[3][4]={2.4,1.05,2.5,0.38,3.0,1.12,2.5,0.35,3.6,1.20,2.5,0.32};  
    char mode[][15]={"Organic","Semi-Detached","Embedded"};  
    int size = 4;  
    calculate(table,3,mode,size);  
    return 0;  
}
```

Output:

The mode is Organic
Effort = 10.289 Person-Month
Development Time = 6.06237 Months
Average Staff Required = 2 Persons.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Basic Cocomo ...**
 - The **basic Cocomo model** assumes that **the effort is only a function of the number of lines of code** and some **constants evaluated** according to the **different software systems**.
 - However, **in reality, no system's effort** and **schedule** can be solely calculated on the basis of **Lines of Code**.
 - For that, various other factors such as **reliability, experience, Capability**. These factors are known as **Cost Drivers** and **the Intermediate Model** utilizes **15 such drivers for cost estimation**.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Intermediate Model ...**
 - Classification of Cost Drivers and their attributes:
 - (i) Product attributes –**
 - Required software reliability extent
 - Size of the application database
 - The complexity of the product
 - (ii) Hardware attributes –**
 - Run-time performance constraints
 - Memory constraints
 - The volatility of the virtual machine environment
 - Required turn about time.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Intermediate Model ...**
 - Classification of Cost Drivers and their attributes:
 - (iii) Personnel attributes –**
 - Analyst capability
 - Software engineering capability
 - Applications experience
 - Virtual machine experience
 - Programming language experience
 - (iv) Project attributes –**
 - Use of software tools
 - Application of software engineering methods
 - Required development schedule



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

○ **Intermediate Model ... : Cost Drivers**

Cost Drivers	Very Low	Low	Nominal;	High	Very High
Product Attributes					
Required Software Reliability	0.75	0.88	1.00	1.15	1.40
Size of Application Database		0.94	1.00	1.08	1.16
Complexity of The Product	0.70	0.85	1.00	1.15	1.30



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

○ **Intermediate Model ... : Cost Drivers**

Cost Drivers	Very Low	Low	Nominal;	High	Very High
--------------	----------	-----	----------	------	-----------

Hardware Attributes

Runtime Performance Constraints			1.00	1.11	1.30
Memory Constraints			1.00	1.06	1.21
Volatility of the virtual machine environment		0.87	1.00	1.15	1.30
Required turnabout time		0.94	1.00	1.07	1.15



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

○ **Intermediate Model ... : Cost Drivers**

Cost Drivers	Very Low	Low	Nominal;	High	Very High
--------------	----------	-----	----------	------	-----------

Personnel attributes

Analyst capability	1.46	1.19	1.00	0.86	0.71
Applications experience	1.29	1.13	1.00	0.91	0.82
Software engineer capability	1.42	1.17	1.00	0.86	0.70
Virtual machine experience	1.21	1.10	1.00	0.90	
Programming language experience	1.14	1.07	1.00	0.95	



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

○ **Intermediate Model ...:** Cost Drivers

Cost Drivers	Very Low	Low	Nominal;	High	Very High
Project Attributes					
Application of software engineering methods	1.24	1.10	1.00	0.91	0.82
Use of software tools	1.24	1.10	1.00	0.91	0.83
Required development schedule	1.23	1.08	1.00	1.04	1.10



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Intermediate Model ... : Cost Drivers**
 - **The project manager** is to rate these **15 different parameters** for a particular project on a **scale of one to three**.
 - Then, depending on these ratings, **appropriate cost driver values** are taken from the **above table**.
 - These **15 values** are then **multiplied to calculate the EAF (Effort Adjustment Factor)**.
 - **The Intermediate COCOMO** formula now takes the form:
 - **$E = (a(KLOC)^b) * EAF$**



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Intermediate Model ...: Cost Drivers**
 - The **values of a** and **b** in case of the **intermediate model** are as follows:

Software Projects	a	b
Organic	3.2	1.05
Semi Detached	3.0	1.12
Embeddedc	2.8	1.20

Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity



- **Detailed Model**

- Detailed **COCOMO** incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the software engineering process.
- The **detailed model** uses different effort multipliers for each cost driver attribute.
- In **detailed cocomo**, the whole software is divided into different modules and then we apply **COCOMO** in different modules to estimate effort and then sum the effort.



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **Detailed Model ...**
 - The Six phases of detailed COCOMO are:
 - Planning and requirements
 - System design
 - Detailed design
 - Module code and test
 - Integration and test
 - Cost Constructive model
- **The effort** is calculated as a **function of program size** and a **set of cost drivers** are given according **to each phase of the software lifecycle**.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **LOC-based models ...**

- **Putnam's SLIM model**

- Another set of LOC-based models uses standard distributions as the basis for modeling the phase distribution of effort. **Putnam's SLIM model**, for example, determines the **life cycle effort (K)** in **worker years** based on **number of source statements (S)** (Putnam, 1978):

$$K = S^3 C^{-3} t_d^{-4}$$

- Where **t_d** represents the time of **peak manpower deployment** and **C** is a **technology constant**.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **LOC-based models ...**
 - **Putnam's SLIM model ...**

$$K = S^3 C^{-3} t_d^{-4}$$

- **SLIM** uses the **Rayleigh curve** to model the **distribution** of effort **over time**.
 - A **criticism** of the **LOC-based models** is that they **require estimating LOC before development** begins.
 - However, **accurate LOC estimates may not be available until** after the **detail design is complete**.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **LOC-based models ...**
 - **Putnam's SLIM model ...**

$$K = S^3 C^{-3} t_d^{-4}$$

- The focus on LOC as an indicator of size also leads to problems when a model calibrated for **one coding language** is used for **another without recalibration**.
- Due to this the **variations** in **line counting methods** may change **LOC** by a **wide margin**.

Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity



- **LOC-based models ...**
 - **Function point-based models**
 - The **function point method** first assigns a **weight** to each **unique input type, output type, logical file, external interface file, and external query handled** by an application to reflect the "**level of complexity.**"
 - The **total score** for all **function types**, called **the function count**, and is then modified using the **total ratings (TR)** of **14 processing complexity characteristics** to account for the **different kinds of system requirements and development environments.**



Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity

- **LOC-based models ...**
 - **Function point-based models ...**
 - Although **the function point approach** can provide an **early estimate of size**, it also has **certain problems**.
 - The **two dominant problems** associated with this **metric** involve the **effort required to collect function point data** and the **difficulty in obtaining consistent estimates from multiple individuals**.
 - *Reference Paper about the **estimation of Function points** on PP. 166-167: Sangeetha, K., & Dalal, P. (2015). A review paper on software effort estimation methods. no, 3, 163-169.*

Week 12, 13 and 14 Lessons:

Requirements Effort Estimation & Project Velocity



- **Estimating in Agile methodologies**
 - Estimations in Agile projects are derived collaboratively by the project team or its representation.
 - Knowledge of a project team (or teams) **velocity** is necessary to estimate **the attributes of the project** in Agile methodologies.
 - The **velocity** is a measure of a team's rate of progress; it is calculated by summing the **estimations assigned to each features** delivered by the **team** during the **single iteration**.



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Estimating in Agile methodologies ...**
 - When the **velocity** and a **project size** are **known**, the **duration of the project**, as a **number of iterations**, could be calculated by dividing a **sum of all features estimations** by **team velocity**.
Duration = Size/Velocity,
 - Where **Size** - a size of a project in measure unites, **Velocity** - a project team velocity in measure unites per an iteration



Week 12, 13 and 14 Lessons: Requirements Effort Estimation & Project Velocity

- **Estimating in Agile methodologies ...**
 - The effort of a project is received using the following formula

$$Effort = \frac{Size \cdot TeamNumber \cdot IterationLength}{Velocity}, \text{ man-days}$$

- Where **Size** - a size of a project in measure unites, **TeamNumber** - a project team number, **Iteration Length** - an iteration length in man-days, **Velocity** - a project team velocity in measure unites per an iteration.